Time: 30 minutes Max marks: 10

Instructions:
- Do not plagiarize. Do not assist your classmates in plagiarism.
- Show your full solution for the questions to get full credit.
- Attempt all questions that you can.
- In the unlikely case a question is not clear, discuss it with an invigilating TA. Please ensure that you clearly include any assumptions you make, even after clarification from the invigilator.

Imp. Note: Q1 has *negative marking* - negative 0.5 points for each wrong answer.

1. (3 points) Match the relative camera positions with the corresponding epipolar lines. {**Important Note:** Each correct match will be awarded 0.5, with a bonus of 0.5 if all are correctly answered. However, there is negative 0.5 points awarded for each wrong match.}

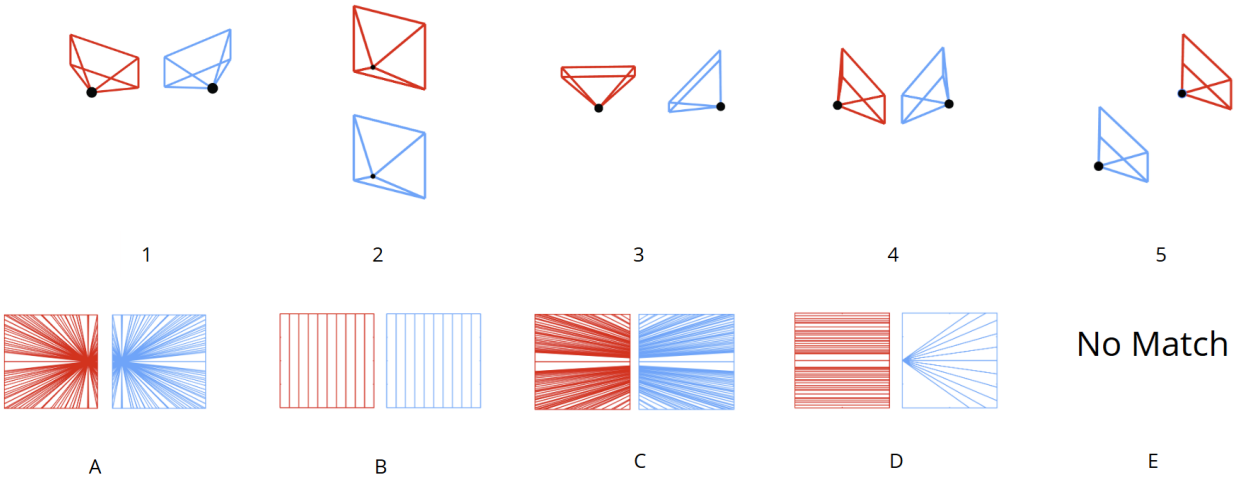

Figure 1: Q1 - Match the relative camera orientation with the corresponding epipolar lines.

Solution:

Rubric:
+0.5 for each correct (0.5*5=2.5), if all correct (+0.5 Bonus) N
-0.5 for each wrong match

The simplest way to solve this is by connecting the camera centers and checking where does the *baseline* intersect with the image plane. This point of intersection gives us the epipole and the epipolar lines intersect at the epipole (even when they are at infinity!).

$$1-C \quad 2-B \quad 3-D \quad 4-A \quad 5-E$$

2. (2 points) Why is stereo rectification needed (provide 2 reasons)? How is it achieved? You may assume that the stereo setup is calibrated, i.e., intrinsics of both cameras as well as the extrinsics (relative rotation and translation) are known.

Solution:
Rubric:
+0.25 point if the explanation states via a homography;
+0.25 if it states that the homography is constructed via pure rotations;
+0.5 more if it also mentions aligning the X, Y, and Z axes to be all parallel; equivalently, it can be said that the epipoles are at infinity along the X-axis or that the X-axes of both virtual images are aligned with the translation vector.

Why is stereo rectification needed?

1. Dimensionality Reduction: In a standard stereo setup, a matching point could be anywhere in the second image (a 2D search). Rectification aligns the images so that a point in the left image always lies on the exact same horizontal row (scanline) in the right image. This reduces the search space from 2D to a 1D horizontal line.
2. Efficiency and Speed: Searching along a single row is significantly faster and computationally cheaper, making real-time applications like autonomous driving or robot navigation possible.

How is it achieved?

Rectification is achieved by independently applying homographies to map the respective images from the two cameras to *virtual* image planes via a pure rotation, such that the corresponding *virtual* cameras have parallel optical axes. Moreover, the horizontal and vertical axes in the image planes should also be aligned with the X-axis parallel to the baseline.

3. (2 points) Transformer architectures.
   (a) (1 point) In a CNN, the "receptive field" (the area of the image the network *sees*) grows slowly as you go deeper into the layers. In a ViT, what is the size of the receptive field in the very first layer?
   (b) (1 point) In ViT, we break an image into a grid of fixed-size patches (e.g., pixels) and treat them as "tokens". If two pixels are in the same patch, they are processed together immediately. If two pixels are in adjacent patches, how does the Transformer model *know* they are neighbors compared to two pixels on opposite sides of the image?

Solution:
Rubric:
1 mark each for correct reason

(a) (1 point) The receptive field of every self-attention layer is the *entire image* as every token (corresponding to a patch) attends to every other token (patch) of the image.

(b) (1 point) The transformer model utilizes a positional encodings to infer the proximity of two patches.

4. (3 points) Explain the steps of how multi-head self-attention is computed in the Transformer model.

Solution:
Rubric:
+1 Linear Projection
+1 Scaled Dot Product Attention
+1 Concatenation and Final Projection

1. Linear Projection: For each head i, the input X is projected into Query (Q), Key (K), and Value (V) subspaces:
$$Q_i = XW_i^Q, \quad K_i = XW_i^K, \quad V_i = XW_i^V \tag{1}$$
where $W_i^Q \in \mathbb{R}^{d \times d_Q}$, $W_i^K \in \mathbb{R}^{d \times d_K}$, and $W_i^V \in \mathbb{R}^{d \times d_V}$, where d is the dimensionality of the token embedding, and $d_Q = d_K = d_V$ are the dimensionality of the query, key and value embeddings.

2. Scaled Dot-Product Attention Each head computes attention independently using the scaled dot-product formula:
$$\text{Head}_i = \text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_K}}\right) V_i \tag{2}$$

3. Concatenation and Final Projection The outputs of all h heads are concatenated and projected one last time to integrate the information:
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{Head}_1, \ldots, \text{Head}_h) W^O \tag{3}$$
where $W^O \in \mathbb{R}^{hd_V \times d}$ is the output parameter matrix.